

Identificação da equipe:

Enrico Augusto Pereira Naba

Rodrigo Santos Graça

Victor Luiz Koba Batista

Vitório Fraga Motta

PROJETO CONEXÕES SEGURAS - RELATÓRIO TÉCNICO

Módulo 1 - Sniffing e Interceptação (Camada 2 e 3)

Modo Promíscuo e Captura de Pacotes:

Em condições normais, uma NIC (Network Interface Controller) opera no modo padrão, o driver da placa descarta qualquer quadro Ethernet cujo endereço MAC de destino não corresponda ao da própria interface ou a endereços de broadcast/multicast registrados. Esse comportamento é implementado diretamente no controlador de hardware e no driver do sistema operacional como uma otimização, evitando que a CPU processe tráfego irrelevante.

Quando o modo promíscuo é ativado, via chamada de sistema como *ioctl* com flag *IFF_PROMISC* no Linux, ou pela API *pcap* (usada pelo Wireshark e tcpdump), o driver desabilita este filtro de MAC. A NIC passa a aceitar e encaminhar para a pilha de rede todos os quadros que trafegam no segmento, independentemente do destinatário. Em redes baseadas em hub (topologia de barramento lógico), isso expõe todo o tráfego do segmento. Em redes comutadas (switch), o modo promíscuo por si só captura apenas o tráfego destinado ao host ou broadcast.

Do ponto de vista da segurança, a ativação não autorizada do modo promíscuo é um indicador clássico de comprometimento de um host, ferramentas de detecção como o *ifconfig* ou IDS de rede monitoram essa flag.

Módulo 2 - Criptografia e Handshake (A Base do TLS)

Criptografia Híbrida no HTTPS:

O HTTPS não utiliza criptografia assimétrica (RSA ou ECC) para cifrar o corpo das requisições porque algoritmos assimétricos são computacionalmente custosos, sua segurança depende da dificuldade matemática de fatorar números grandes (RSA) ou resolver o problema do logaritmo discreto em curvas elípticas (ECC). Cifrar grandes volumes de dados com esses algoritmos tornaria o protocolo inviável em termos de desempenho, consumindo muito mais CPU e introduzindo latência significativa.

A solução adotada pelo TLS é a criptografia híbrida, que combina o melhor dos dois mundos:

Handshake (assimétrica):

Durante o handshake TLS, o servidor apresenta seu certificado digital contendo sua chave pública. O cliente usa essa chave pública para cifrar (RSA) ou para realizar um acordo de chaves via ECDHE (Elliptic-Curve Diffie-Hellman Ephemeral), estabelecendo de forma segura um segredo compartilhado. A chave privada do servidor, que nunca trafega pela

rede, é a única capaz de decifrar ou completar o acordo. O resultado é um segredo de sessão: uma chave simétrica temporária.

Transferência de dados (simétrica):

Com a chave de sessão estabelecida, todo o tráfego subsequente é cifrado com AES. A criptografia simétrica são ordens de magnitude mais rápida que a assimétrica e adequada para volumes arbitrários de dados. O modo GCM (Galois/Counter Mode) adiciona autenticação integrada (AEAD), garantindo confidencialidade e integridade simultaneamente.

O uso de ECDHE garante adicionalmente Perfect Forward Secrecy (PFS): mesmo que a chave privada do servidor seja comprometida no futuro, sessões passadas não podem ser decifradas, pois as chaves efêmeras são descartadas após cada sessão.

Módulo 3 - Segurança na Camada de Aplicação

Gestão de Sessão e Cookies (SessionID):

HTTP é um protocolo Stateless, cada requisição é independente. Para manter o estado de autenticação entre requisições, aplicações web utilizam cookies de sessão: o servidor gera um identificador único (SessionID) após a autenticação, armazena o estado associado no servidor (memória, banco de dados ou cache) e envia o SessionID ao cliente via cabeçalho *Set-Cookie*. O navegador reenvia esse cookie automaticamente em todas as requisições subsequentes ao domínio.

Do ponto de vista do desenvolvedor, os principais riscos e preocupações são:

- **Session Hijacking:** Se o SessionID for interceptado (via sniffing em HTTP ou XSS), o atacante pode assumir a sessão do usuário legítimo sem precisar de credenciais. Por isso, o SessionID deve ter entropia suficiente (mínimo 128 bits de aleatoriedade criptograficamente segura).
- **Atributo *HttpOnly*:** Impede que JavaScript acesse o cookie via *document.cookie*, mitigando roubo por XSS.
- **Atributo *Secure*:** Instrui o navegador a enviar o cookie somente em conexões HTTPS, impedindo transmissão em HTTP claro.
- **Atributo *SameSite*:** Controla o envio do cookie em requisições cross-origin, mitigando ataques CSRF (*SameSite=Strict* ou *SameSite=Lax*).
- **Tempo de expiração:** Sessões devem ter tempo de vida limitado e ser invalidadas no servidor após logout, destruir apenas o cookie no cliente não é suficiente.
- **Rotação de SessionID:** Após elevação de privilégio (ex: login), o SessionID deve ser regenerado para evitar Session Fixation.

O Papel do HTTPS na Proteção de Cookies:

O HTTPS protege cookies de sessão em trânsito por meio da camada TLS: os cabeçalhos HTTP são cifrados antes de sair do cliente. Um atacante posicionado na rede (man-in-the-middle) visualiza apenas metadados do protocolo TLS (IP, porta, SNI), mas não o conteúdo dos cabeçalhos.

Se a aplicação operar apenas em HTTP, os cookies trafegam em texto plano. Qualquer atacante no mesmo segmento de rede (rede Wi-Fi pública, rede corporativa comprometida) utilizando ferramentas de captura como Wireshark ou tcpdump consegue ler o SessionID

diretamente dos pacotes capturados e utilizá-lo para impersonar o usuário, ataque conhecido como session hijacking via sniffing. A combinação de HTTP com o atributo *Secure* ausente no cookie representa uma superfície de ataque crítica, especialmente em redes não confiáveis.

Módulo 4 - VPN (Virtual Private Network)

Fundamentos e Funcionamento:

Uma VPN estabelece um túnel cifrado entre o cliente e um gateway de rede, encapsulando o tráfego original dentro de um protocolo seguro. Do ponto de vista da camada de rede, o host cliente passa a operar como se estivesse diretamente conectado à rede privada remota, independentemente de sua localização física.

Os principais protocolos modernos são:

- **WireGuard:** Protocolo moderno (integrado ao kernel Linux desde 5.6), baseado em ChaCha20 para cifração, Poly1305 para autenticação e Curve25519 para troca de chaves. Código reduzido (cerca de 4.000 linhas), superfície de ataque mínima e performance superior.
- **OpenVPN:** Baseado em TLS/SSL, altamente configurável e com suporte amplo. Opera sobre UDP ou TCP, com autenticação por certificados X.509.
- **IPSec/IKEv2:** Protocolo de camada 3 nativo em sistemas operacionais, frequentemente usado em VPNs corporativas. IKEv2 gerencia a negociação de chaves; IPSec (ESP) provê cifração e autenticação dos pacotes IP.

Do ponto de vista corporativo, VPNs garantem que colaboradores remotos acessem recursos internos (servidores, bancos de dados, sistemas ERP) sem expô-los à internet pública, reduzindo drasticamente a superfície de ataque da organização. A autenticação mútua por certificados e a rotação automática de chaves de sessão são práticas essenciais em implantações corporativas.

